MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 2

SERIAL COMMUNICATION BETWEEN ARDUINO AND PHYTON

(MCTA 3203)

SECTION 1

GROUP 6

NO.	MATRIC NO.	NAME
1	2414523	MUHAMMAD RASYAD BIN KHAIRIL ANNUAR
2	2414509	MUHAMMAD HAZIQ HAIKAL BIN MOHD YUSRI
3	2415329	MUHAMMAD IRFAN BIN AMIR AMRAN
4	2410303	MUHAMMAD NUR IKHWAN BIN ABDUL MUKTI

SUBMISSION DATE: 19 OCTOBER 2026

Abstract

This experiment investigates serial communication between an Arduino and a computer using Python. The experiment consists of reading potentiometer values, controlling a servo motor, and integrating an LED and real-time graphical visualization. In the first experiment, potentiometer readings are transmitted from the Arduino to Python through a USB serial connection, and an LED is controlled based on the potentiometer value. In the second experiment, Python is used to control the angle of a servo motor, followed by modifications to enable real-time servo control using a potentiometer. An LED is also integrated to provide visual feedback, while the Matplotlib library is used to plot the potentiometer readings in real time. These experiments demonstrate how Arduino hardware and Python software can work together to perform data communication, monitoring, and control applications.

Table of Contents

1.0 Introduction & Theory

1.1 Materials and Equipment

1.2 Experimental Setup

1.3 Methodology & Programming Logic

1.4 Data Collection & Analysis

1.5 Results & Discussion

1.6 Conclusion

Recommendations

References

Appendices

Acknowledgement

Student Declaration

1.0 Introduction & Theory

Serial communication allows data to be exchanged between an Arduino board and a computer through a USB connection. In this experiment, Python uses the PySerial library to receive data from the Arduino and send commands to control external components.

A potentiometer acts as an adjustable input device that provides analog readings ranging from 0 to 1023 on a typical 10-bit Arduino analog input. These readings are transmitted to Python and used to control an LED based on a specified threshold. The experiment also involves controlling a servo motor's angle between 0° and 180° using Python commands and potentiometer input. Additionally, the Matplotlib library is used to display potentiometer readings in real time through graphical visualization.

Overall, this experiment demonstrates the application of serial communication, sensor input, actuator control, and data visualization in a basic mechatronic system.

1.1 Materials and Equipment

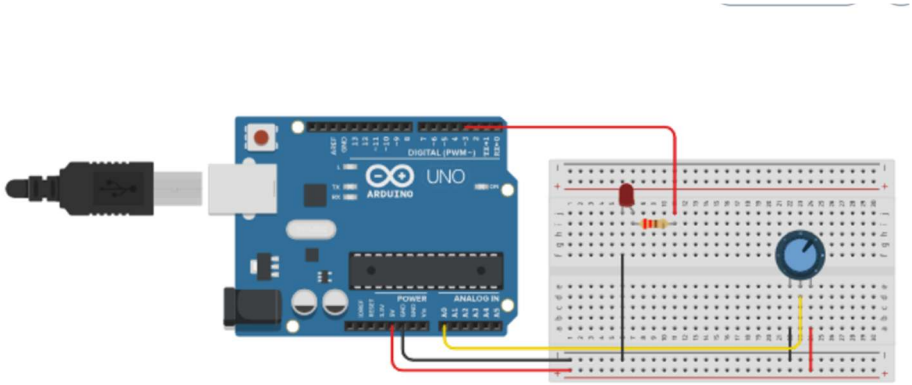
Hardware:

- Arduino board and USB cable
- Potentiometer
- Servo motor
- LED
- 220 Ω resistor
- Breadboard
- Jumper wires
- Computer

Software and libraries:

- Arduino IDE
- Python
- PySerial library for serial communication
- Matplotlib library for graphical visualization
- Arduino Servo library for servo motor control

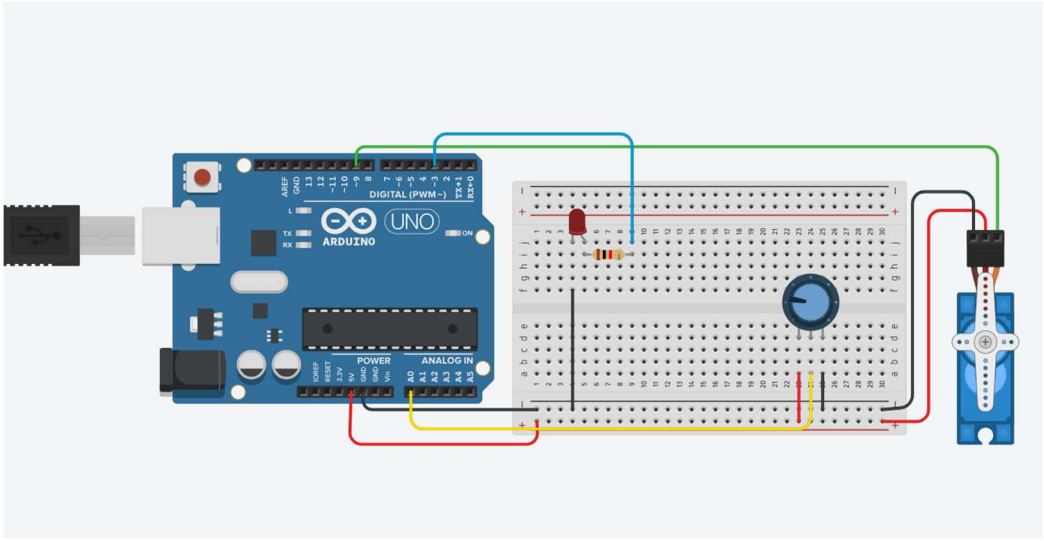
1.2 Experimental Setup



Circuit Design Experiment 1 Schematics using Tinkercad

Experiment 1

- The circuit consists of an Arduino board, a potentiometer, an LED, and a 220 Ω resistor. The potentiometer is connected to analog pin A0, while the LED is connected to digital pin 3 through the resistor. The Arduino is connected to a computer via USB to transmit potentiometer readings to Python through serial communication.



Circuit Design Schematics Experiment 2 using Tinkercad

Experiment 2

- The circuit consists of an Arduino board, a potentiometer, a servo motor, an LED, and a 220 Ω resistor. The potentiometer is connected to analog pin A0, the servo signal wire to digital pin 9, and the LED to digital pin 3 through the resistor. The servo motor's position is controlled using potentiometer readings or commands received through serial communication with Python.

1.3 Methodology & Programming Logic

Experiment 1

i) Experiment Steps:

1. The Arduino was connected to the computer via a USB cable.
2. The sketch was uploaded to the Arduino using the Arduino IDE.
3. The Python script was run on the computer.
4. As the potentiometer knob was turned, the potentiometer readings were displayed in the Python terminal.
5. These readings can be used for various experiments, data logging, or control applications, depending on the project requirements.
6. The Serial Plotter was opened by going to Tools -> Serial Plotter in the Arduino IDE.
7. The COM port which the Arduino was connected was selected in the Serial Plotter.

8. The baud rate in the Serial Plotter was ensured to match the one set in the Arduino code (e.g., 9600).
9. As the potentiometer knob was turned, real-time potentiometer readings were displayed in the Serial Plotter, creating a graphical representation of the data, and changes in the values were observed as the potentiometer was adjusted.
10. The Serial Plotter was customized by adjusting settings such as the graph range, labels, and colors.
11. The circuit was extended by adding an LED in series with a 220 Ω resistor, and the Python code was updated so that the LED was turned ON automatically when the potentiometer value exceeded half of its maximum range and was turned OFF otherwise.

ii) Arduino Code (read the potentiometer value):

```
void setup() {
Serial.begin(9600);
}
void loop() {
int potValue = analogRead(A0);
Serial.println(potValue);
delay(1000); // add a delay to avoid sending data too fast
}
```

iii) Phyton Script (read the potentiometer value):

```
import serial
ser = serial.Serial('COM3', 9600)
try:
  while True:
    pot_value = ser.readline().decode().strip()
    print("Potentiometer Value:", pot_value)

except KeyboardInterrupt:
  ser.close()
  print("Serial connection closed.")
```

Arduino Code (LED):

```
const int led=3;

void setup() {

pinMode(led,OUTPUT);

Serial.begin(9600);
}
void loop() {
int potValue = analogRead(A0);
Serial.println(potValue);

delay(1000);
if(potValue>511)
  {
    digitalWrite(led,HIGH);
  }
else
{
     digitalWrite(led,LOW);

}

}
```

Experiment 2

Experiment steps:

1. The circuit was constructed as shown in **Circuit Design Schematics Experiment 2 using Tinkercad** diagram (Signal-pin 9, Vcc-5V, GND-GND).

2. The Arduino code that reads angle data from the serial port and moves the servo accordingly was written:

```
#include <Servo.h>
Servo myservo; // to control a servo
int pos = 0; // variable to store the servo position
```

```
void setup() {
myservo.attach(9); // attaches the servo on pin 9
}
void loop() {
for (pos = 0; pos <= 180; pos += 1) {

// goes from 0 to 180 degrees

myservo.write(pos);

delay(15); // waits 15ms for the servo

// to reach the position

}
for (pos = 180; pos >= 0; pos -= 1) {

// goes from 180 to 0 degrees

myservo.write(pos);

// tell servo to go to position in
// variable 'pos'

delay(15); // waits 15ms for the servo
// to reach the position

}
}
```

3. The code was uploaded to the Arduino board.
4. A new Python script was created by using a code editor:

```
import serial

ser = serial.Serial('COM3', 9600)
try:
    while True:
        angle = input("Enter servo angle (0-180 deg, or 'q' to quit): ")
        if angle.lower() == 'q':
            break
        angle = int(angle)
        if 0 <= angle <= 180:

            ser.write(str(angle).encode())
        else:
            print("Angle must be between 0 and 180 degrees.")
except KeyboardInterrupt:
    pass
finally:
    ser.close()
    print("Serial connection closed.")
```

5. The Python script above was saved with a .py extension

6. The Python script was run, which prompted the user to enter the servo angle, and an angle between 0 and 180 degrees was entered to control the servo.

7. When the experiment was completed, 'q' was entered to exit the Python script.

8. The Arduino and Python code were enhanced to incorporate a potentiometer for real-time adjustments of the servo motor's angle. It was ensured that, in the updated Arduino code, execution could be halted by pressing a designated key on the computer's keyboard. Following the modification, the Python script was restarted to receive and display servo position data from the Arduino over the serial connection. While the potentiometer was experimented with, corresponding changes in the servo motor's position were observed.

9. The experiment was enhanced by adding an LED that was lit based on the potentiometer value or servo position, and a further layer of control and feedback was added.

Arduino code:

```
#include <Servo.h>

Servo myservo;
const int potPin = A0;  // Potentiometer connected to analog pin A0
const int ledM = 3;

void setup() {
  Serial.begin(9600);   // Initialize serial communication
  myservo.attach(9);    // Attaches the servo on pin 9
}

void loop() {
  int potValue = analogRead(potPin);              // Read potentiometer (0 to 1023)
  int angle = map(potValue, 0, 1023, 0, 180);  // Convert to 0-180 degrees

  myservo.write(angle);                    // Move the servo

  // Send the angle to the Python script
  Serial.println(angle);

  delay(100);
  if (potValue > 511)
  {
    digitalWrite(ledM,HIGH);
  }

  else
  {
    digitalWrite(ledM,LOW);
  }
}
```

10. Graphical visualization using matplotlib was integrated to plot potentiometer values in real-time.

Python script:

```
import serial
import matplotlib.pyplot as plt
ser = serial.Serial('COM3', 9600)

plt.ion() # Interactive mode for real-time plot
fig, ax = plt.subplots()
x_vals, y_vals = [], []
try:
    while True:
        pot_value = ser.readline().decode().strip()
        print("Potentiometer Value:", pot_value)
        x_vals.append(len(x_vals))
        y_vals.append(int(pot_value))
        ax.clear()
        ax.plot(x_vals, y_vals) # Plot potentiometer readings
        plt.pause(0.1) # Update plot
except KeyboardInterrupt:
    ser.close()
    plt.ioff() # Turn off interactive mode
    plt.show() # Final plot display
    print("Serial connection closed.")
```

1.4 Data Collection & Analysis

Experiment 1:

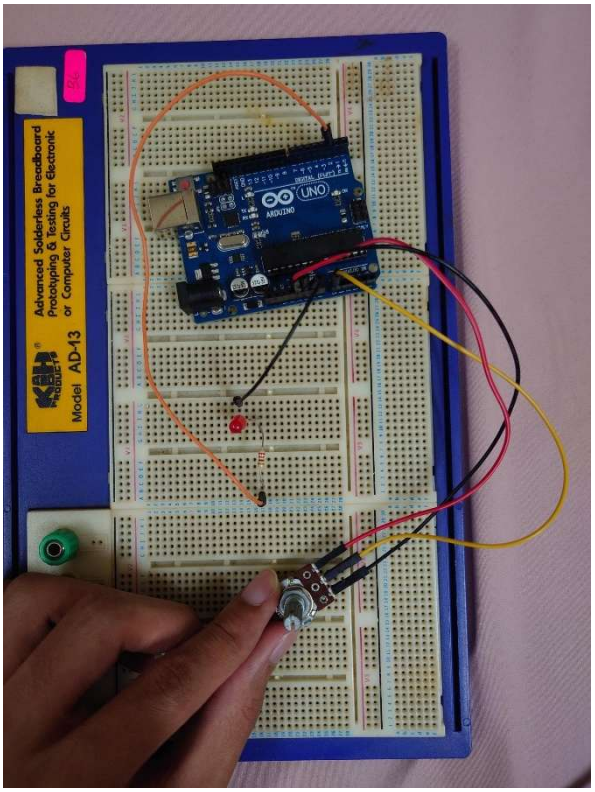
- Potentiometer readings were displayed on both Arduino serial monitor and Python terminal.
- Readings change when the potentiometer knob is turned.
- When LED is added to the circuit, it turns ON once the potentiometer value exceeds half of its maximum range, which shows that the code that we modified is functioning correctly to perform the task.

Experiment 2:

- The servo rotates based on the angle we entered.
- The experiment ends when we ‘q’ is pressed.
- An LED lights up when the value of potentiometer or servo angle exceeds half of its maximum range indicates that the modified code is working well.
- Matplotib plotted potentiometer values in real-time.

1.5 Results & Discussion

Experiment 1

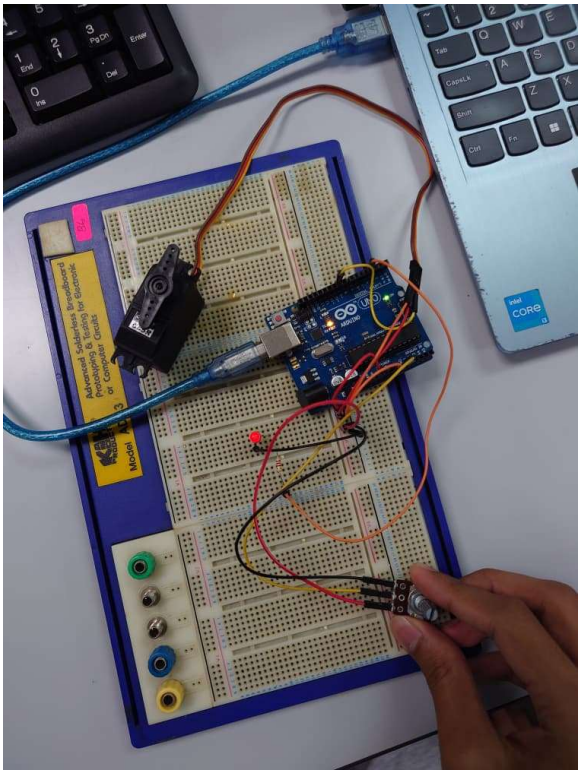


The experimental results showed that LED can be automatically turned ON and OFF using a potentiometer.

Using the Arduino application, the increment and decrement of the reading of the potentiometer can be seen through the serial monitor. We have set up the midpoint which is 511 to be the point where the LED will turn ON or OFF.

Once the value increases above the midpoint, the LED will turn ON. The same goes to the decreasing value where when it goes below midpoint, the LED will turn OFF.

Experiment 2



The experimental results showed that potentiometer can control the angle of servo motor rotation. A continuous graph then will be automatically plotted by matplotlib following the increment or decrement of the angle of servo motor.

The coding in Arduino application will get the values of the potentiometer. The values then will be converted into angle and the data will be sent to Arduino board to be processed. LED then will light up once the value increases above the midpoint which is 511. The LED also will be turned off once the value decreases below the midpoint value.

Meanwhile, the coding in Python application will get the values from the Arduino board and will connect to the matplotlib. Matplotlib will automatically open and receive data from Arduino board and begin to plot a continuous graph.

1.6 Conclusion

In conclusion, the experiments successfully demonstrated serial communication between an Arduino board and Python for data monitoring and hardware control. In Experiment 1, the potentiometer readings were successfully transmitted from Arduino to Python and displayed in real time. The LED was also controlled based on the potentiometer value, turning ON when the reading exceeded the midpoint of 511 and turning OFF when the value was below the threshold.

In Experiment 2, the potentiometer was successfully used to control the servo motor's rotation angle between 0° and 180°. The integration of Python and the Matplotlib library also allowed the potentiometer readings to be displayed graphically in real time. Overall, both experiments demonstrated how Arduino and Python can work together to perform data acquisition, processing, monitoring, and control. These experiments also provided a better understanding of serial communication and its applications in mechatronic systems.

Recommendations

- Reduce Data Transmission Delay
  Reduce the delay in the Arduino code to allow faster data transmission and smoother real-time monitoring and improve the responsiveness of the LED and servo motor when adjusting the potentiometer.

- Improve Circuit Connections
  Ensure all circuit connections are properly secured to prevent loose connections and unstable readings and apply filtering techniques such as averaging multiple potentiometer readings to reduce noise and achieve smoother servo motor movement.

- Enhance Python Programming
  Implement error-handling features to prevent the Python program from crashing when the serial connection is interrupted and improve the graphical visualization by adding proper axis labels and a fixed data range.

- Implement Data Logging

  Add a data logging feature to record potentiometer readings over time and save the collected data for further analysis and evaluation of the system's performance.

References

- Autodesk. (n.d.). *Tinkercad circuits*. https://www.tinkercad.com/circuits
- Arduino. (n.d.). *Arduino IDE*. https://www.arduino.cc/en/software
- Matplotlib

Appendices

- Arduino Codes

```
void setup() {
Serial.begin(9600);
}
void loop() {
int potValue = analogRead(A0);
Serial.println(potValue);
delay(1000); // add a delay to avoid sending data too fast
}
```

```
const int led=3;

void setup() {

pinMode(led,OUTPUT);

Serial.begin(9600);
}
void loop() {
int potValue = analogRead(A0);
Serial.println(potValue);

delay(1000);
if(potValue>511)
  {
    digitalWrite(led,HIGH);
  }
else
{
      digitalWrite(led,LOW);

}

}
```

```
#include <Servo.h>

Servo myservo;
const int potPin = A0;  // Potentiometer connected to analog pin A0
const int ledM = 3;

void setup() {
  Serial.begin(9600);   // Initialize serial communication
  myservo.attach(9);    // Attaches the servo on pin 9
}

void loop() {
  int potValue = analogRead(potPin);              // Read potentiometer (0 to 1023)
  int angle = map(potValue, 0, 1023, 0, 180);  // Convert to 0-180 degrees

  myservo.write(angle);                        // Move the servo

  // Send the angle to the Python script
  Serial.println(angle);

  delay(100);
  if (potValue > 511)
  {
    digitalWrite(ledM,HIGH);
  }

  else
  {
    digitalWrite(ledM,LOW);
  }
}
```

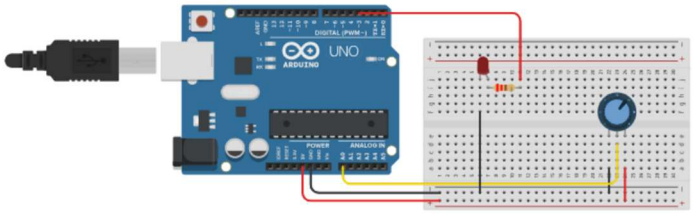
- Python Codes

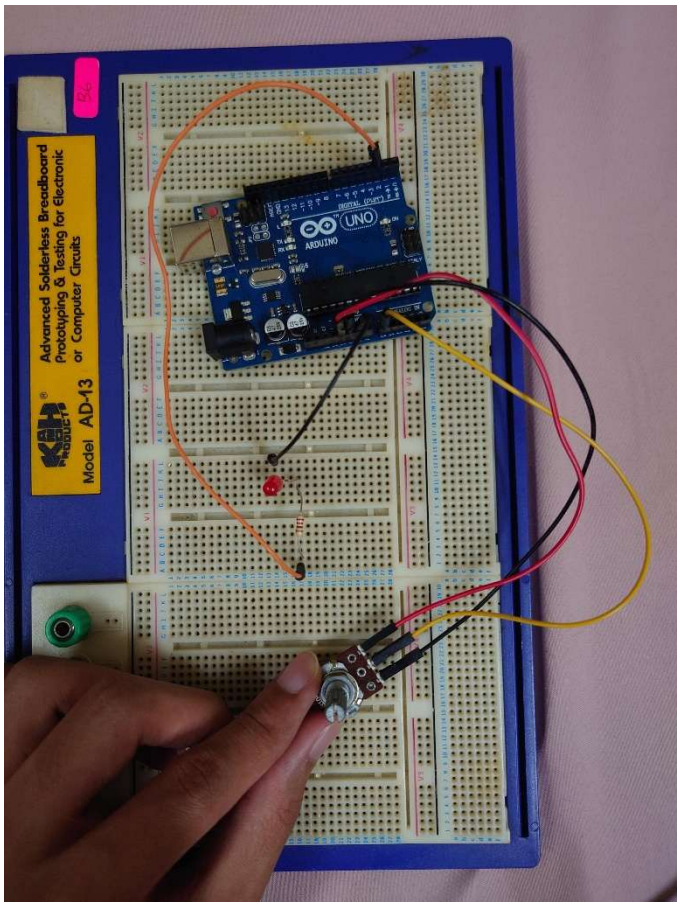
```
>>> import serial
... ser = serial.Serial('COM3', 9600)
... try:
...     while True:
...         pot_value = ser.readline().decode().strip()
...         print("Potentiometer Value:", pot_value)
...
... except KeyboardInterrupt:
...     ser.close()
...     print("Serial connection closed.")
```

```
>>> import serial
... import matplotlib.pyplot as plt
... ser = serial.Serial('COM3', 9600)
...
... plt.ion() # Interactive mode for real-time plot
... fig, ax = plt.subplots()
... x_vals, y_vals = [], []
... try:
...     while True:
...         pot_value = ser.readline().decode().strip()
...         print("Potentiometer Value:", pot_value)
...         x_vals.append(len(x_vals))
...         y_vals.append(int(pot_value))
...         ax.clear()
...         ax.plot(x_vals, y_vals) # Plot potentiometer readings
...         plt.pause(0.1) # Update plot
... except KeyboardInterrupt:
...     ser.close()
...     plt.ioff() # Turn off interactive mode
...     plt.show() # Final plot display
...     print("Serial connection closed.")
```

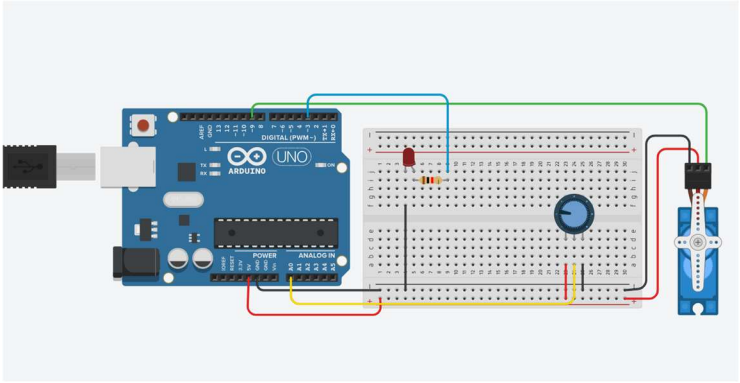
- Circuit Diagram

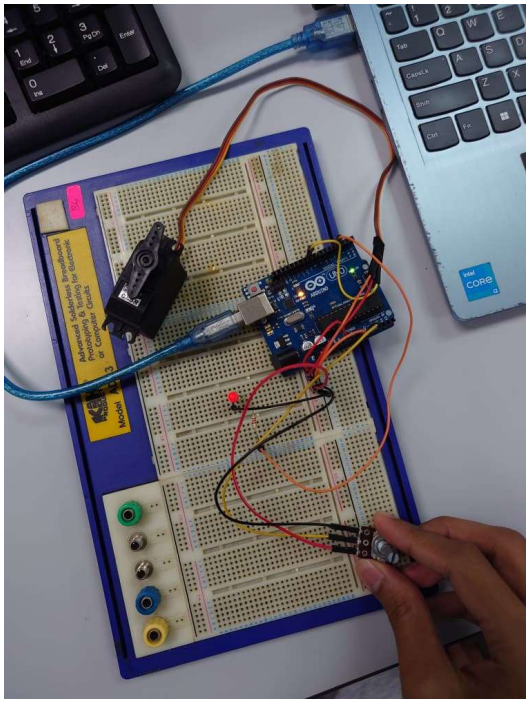
Experiment 1





Experiment 2





Acknowledgement

We would like to express our deepest appreciation to our instructor and laboratory personnel for their guidance, support, and assistance throughout this experiment. Their explanations and feedback helped us understand the practical application of microcontroller programming, sensor communication, and data acquisition. We would also like to thank our group members for their cooperation and contribution during the preparation, assembly, testing, and analysis of the experimental system.

Student's Declaration

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Name: MUHAMMAD RASYAD BIN KHAIRIL ANNUAR Agree **[/]** Matric: 2414523

Name: MUHAMMAD HAZIQ HAIKAL BIN MOHD YUSRI Agree **[/]** Matric: 2414509

Name: MUHAMMAD IRFAN BIN AMIR AMRAN Agree **[/]** Matric: 2415329

Name: MUHAMMAD NUR IKHWAN BIN ABDUL MUKTI Agree **[/]** Matric: 2410303